ASSIGNMENT 5 – CO5

NAME : RAJA RAGAVI R K

REGISTRATION : 1911260241

SUBJECT : PYTHON PROGRAMMING

COURSE CODE : CSA0822

GITHUB LINK: https://github.com/RajaRagavi16/CSA0822-1911260241-

TASK A1: DESIGN THE DATA ACQUISITION

AIM:

Describe the station CSV, design SQLite storage and compare CSV scans with SQL queries.

THEORY:

Use one CSV row per station, date and hour with pollutant values and temperature. Anticipate blank/NA readings, negative and implausibly high sensor faults, duplicate timestamps, Fahrenheit units, mixed date formats, inconsistent station names and nonnumeric cells. Store stations in a lookup table and readings with a composite station/day/hour key and foreign key. Three typical queries are monthly PM2.5 average per station, station comparison for a day, and worst hour. CSV/DataFrame analysis scans O(n) per question and holds data in memory; indexed SQLite is disk-backed and takes about O(log n + matches).

PROGRAM CODE:

```
-- Station table: station_id INTEGER PRIMARY KEY, name TEXT UNIQUE NOT NULL,
-- temperature_unit TEXT CHECK(temperature_unit IN ('C','F'))
-- Reading table uses (station_id, day, hour) as its primary key.
CREATE TABLE stations(station_id INTEGER PRIMARY KEY,name TEXT UNIQUE,unit TEXT);
CREATE TABLE readings(station_id INTEGER,day TEXT,hour INTEGER,pm25 REAL,pm10 REAL,no2 REAL,temp_c
  REAL,
 PRIMARY KEY(station_id,day,hour),FOREIGN KEY(station_id) REFERENCES stations);
CREATE INDEX ix_readings_station_day ON readings(station_id,day);
-- Mapping: CSV station name -> station_id; date -> ISO day; blank/'NA' -> NULL.
SELECT AVG(pm25) FROM readings JOIN stations USING(station_id)
 WHERE name=? AND day BETWEEN ? AND ?;
SELECT name,AVG(pm25) FROM readings JOIN stations USING(station_id)
 WHERE day=? GROUP BY name;
SELECT day,hour,pm25 FROM readings WHERE station_id=? AND pm25 IS NOT NULL;
-- CSV/pandas scans n rows per question (O(n), memory O(n)); indexed SQL is about O(log n + matches),
  disk-backed.
```

OUTPUT:

```
Source: station,date,hour,PM2.5,PM10,NO2,temperature
Quality checks: blank/NA, negative or implausible values, duplicate timestamp,
Fahrenheit stations, inconsistent dates/names/numeric text.
Three board queries: monthly station average; daily station comparison;
worst hour of a day. SQL index supports station and date filtering.
```

TASK A2: DESIGN THE CLEANING AND NORMALISATION

AIM:

Set defensible rules for missing, faulty, duplicate and Fahrenheit readings, then define PM2.5 scaling and standard formats.

THEORY:

Fill an isolated missing PM2.5 value with the station median; forward-fill only a short gap where continuity is meaningful. Keep impossible readings out of analysis and record them for review. Deduplicate by station/date/hour, preferring the latest valid record. Convert Fahrenheit with (F-32)*5/9. Min-max scaling maps values to [0,1] and suits bounded comparisons; z-score centers by mean and standard deviation and suits standardized statistical analysis. If missing values are dropped, the average can be biased when missingness is systematic. Dropping and forward-fill are O(n); median sorting is O(n log n). Use ISO dates, trimmed title-case station names and numeric values with consistent precision.

PROGRAM CODE:

```
import statistics
def clean_value(x):
 if x in ('','NA',None):return None
 v=float(x)
 return v if 0<=v<=999 else None
def fahrenheit_to_c(f):return (f-32)*5/9
def minmax(xs):
 lo,hi=min(xs),max(xs)
 return [round((x-lo)/(hi-lo),3) if hi!=lo else 0.0 for x in xs]
def zscore(xs):
 mu=statistics.mean(xs);sd=statistics.pstdev(xs)
 return [(x-mu)/sd if sd else 0.0 for x in xs]
print('missing -> median for PM2.5; forward-fill only short sensor gaps')
print('negative/>999 -> NULL and quarantine; duplicates -> keep latest valid timestamp')
print('32 F ->',fahrenheit_to_c(32),'C')
print('minmax:',minmax([48,95,121]))
print('zscore:',[round(x,3) for x in zscore([48,95,121])])
print('formats: ISO YYYY-MM-DD, title-case station, numeric floats to 3 decimals')
```

OUTPUT:

```
missing -> median for PM2.5; forward-fill only short sensor gaps
negative/>999 -> NULL and quarantine; duplicates -> keep latest valid timestamp
32 F -> 0.0 C
minmax: [0.0, 0.644, 1.0]
zscore: [-1.324, 0.232, 1.092]
formats: ISO YYYY-MM-DD, title-case station, numeric floats to 3 decimals
```

TASK A3: DESIGN THE ANALYSIS AND VISUALISATION

AIM:

Choose charts for four air-quality questions and define the daily station summary.

THEORY:

Use a line chart for monthly PM2.5 trends, a bar chart to compare stations, a scatter plot for temperature and PM2.5, and hourly box plots or a histogram to expose worst-hour patterns. Label axes, units and titles. Produce the summary by station and date with minimum, maximum, mean and valid count; SQL GROUP BY is concise, while a one-pass Python dictionary gives the same O(n) time. SQL avoids loading every reading into Python; the loop uses O(number of groups) memory.

PROGRAM CODE:

```
-- Daily summary table fields:
-- station_id, day, pollutant, minimum, maximum, mean, valid_count
SELECT station_id, day, MIN(pm25), MAX(pm25), AVG(pm25), COUNT(pm25)
FROM readings GROUP BY station_id, day;
-- chart plan:
-- month PM2.5 trend: line; station comparison: bar;
-- temperature vs PM2.5: scatter; worst-hour distribution: box plot/histogram.
-- Python loop alternative: one pass, dictionary key=(station,day),
-- update count/sum/min/max. Both SQL GROUP BY and loop are O(n) time.
-- SQL streams/groups in the database; Python dictionary uses O(groups) memory.
```

OUTPUT:

```
Trend over month -> line chart (time order)
Compare stations -> bar chart (station categories)
Temperature vs PM2.5 -> scatter (relationship/outliers)
Worst hours -> box plot by hour; histogram for distribution
Summary row: station, date, min, max, mean, valid_count
```

TASK A4: PLAN DEBUGGING AND PROFILING

AIM:

Plan how to find cleaning errors and measure slow sections with Python debugging and profiling tools.

THEORY:

Use print briefly to inspect raw and converted values, assertions for invariants, logging with row and station context, and pdb or IDE breakpoints to step through failures. Measure pipeline elapsed time with perf_counter, small-function repeats with timeit, and call costs with cProfile. Thousands of print calls can slow a run and clutter output; logging levels can be changed or disabled centrally, and debuggers can be detached without changing program logic. Compare equivalent inputs and repeat timings before optimizing.

PROGRAM CODE:

```
# Debugging sequence
assert row['station'] in station_ids
assert value is None or 0 <= value <= 999
# logger = logging.getLogger(__name__); logger.debug('row=%s', row_number)
# breakpoint() or pdb.set_trace() at the failed conversion
# Profiling examples:
# t=time.perf_counter(); clean(rows); print(time.perf_counter()-t)
# timeit.repeat('clean(data)', globals=globals(), number=5)
# cProfile.run('build_database(rows)')
# Log at DEBUG during diagnosis; INFO/WARNING in production; disable DEBUG later.
```

OUTPUT:

```
print -> quick local value check; remove after diagnosis
assert -> executable invariant, e.g. PM2.5 is within sensor bounds
logging -> levels, timestamps and module context; configurable output
pdb/IDE -> breakpoints, step, inspect variables and call stack
perf_counter -> end-to-end elapsed time; timeit -> repeat small functions
cProfile -> cumulative function costs; optimize the slowest measured step
Debug logging can be turned down without editing processing code.
```

TASK A5: DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS

AIM:

Design a negation-aware lexicon scorer, extractive summary and database-backed rule chatbot.

THEORY:

Score each word using positive and negative lexicons; flip its value when preceded by not, never or no. Summarize by selecting sentences containing frequent non-stop words. For a chatbot, recognize station and date keywords, then issue a parameterized SQL query. It handles only known phrasing, cannot infer sarcasm or complex intent, and can expose gaps in the database. A trained Naive Bayes model needs labeled examples and more model memory but can generalize better to varied language; a pretrained language model uses much more memory and compute, with higher potential accuracy. Lexicon scoring is O(words) and tiny in memory.

PROGRAM CODE:

```
import re,collections
POS={'good','clean','fast','helpful','improved'};NEG={'bad','dirty','slow','rude','smoke','broken'};NE
  GATORS={'not','never','no'}
def score(text):
 words=re.findall(r"[a-z']+",text.lower());total=0
 for i,w in enumerate(words):
  v=1 if w in POS else -1 if w in NEG else 0
  if i and words[i-1] in NEGATORS:v=-v
  total+=v
 return total
def summary(text,k=2):
 sents=re.split(r'(?<=[.!?])\s+',text.strip());freq=collections.Counter(w for s in sents for w in
  re.findall(r'[a-z]+',s.lower()) if w not in {'the','and','is','are','to','a','of'})
 return sorted(sents,key=lambda s:-sum(freq[w] for w in re.findall(r'[a-z]+',s.lower())))[:k]
def chatbot(question,con):
 m=re.search(r'pm2\.?5.*in ([a-z ]+)',question.lower())
 if not m:return 'Ask for PM2.5 in a named station and date.'
 row=con.execute('SELECT pm25 FROM readings JOIN stations USING(station_id) WHERE name=? ORDER BY day
  DESC LIMIT 1',(m.group(1).strip().title(),)).fetchone()
 return f'PM2.5: {row[0]}' if row else 'No reading found.'
text='Air is not clean. Smoke is bad. The service improved.'
print('score:',score(text),'summary:',summary(text))
print('chatbot rule: keyword + parameterized SQL; limitations: fixed phrasing, lexicon bias, no
  context')
```

OUTPUT:

```
score: -2 summary: ['Air is not clean.', 'Smoke is bad.']
chatbot rule: keyword + parameterized SQL; limitations: fixed phrasing, lexicon bias, no context
```

TASK B1: IMPLEMENT AND VALIDATE THE CLEANING PIPELINE

AIM:

Clean a small CSV, verify data before and after, store records in SQLite, query results and supply labeled chart code.

THEORY:

The sample tests missing/NA, blank, negative, over-999 and duplicate readings. The cleaner skips invalid rows, converts valid readings to float, normalizes station names and removes duplicates. The min-max guard handles a constant series. Insert with executemany and use parameterized SQL for monthly average and daily grouping. For visualization, use the A3 line, bar, scatter and hourly box plots with labeled axes and titles; interpret the monthly trend and station differences from those plotted values. pandas vectorization is concise but typically uses O(n) memory; csv iteration can stream with O(1) extra row memory.

PROGRAM CODE:

```
import csv,io,sqlite3,statistics
RAW="""station,date,pm25\nAdyar,2026-10-01,48\nAdyar,2026-10-02,NA\nAdyar,2026-10-03,\nGuindy,2026-10
  01,95\nGuindy,2026-10-02,-5\nGuindy,2026-10-03,121\nAdyar,2026-10-01,48\nAdyar,2026-10-05,1001\n"""
def load_clean(text):
 rows=[];seen=set()
 for r in csv.DictReader(io.StringIO(text)):
  try:
   v=float(r['pm25'])
   if not 0<=v<=999:continue
   key=(r['station'].strip().title(),r['date'],v)
   if key not in seen:seen.add(key);rows.append(key)
  except (ValueError,TypeError,KeyError):continue
 return rows
def min_max(v):
 lo,hi=min(v),max(v)
 return [round((x-lo)/(hi-lo),3) if hi!=lo else 0.0 for x in v]
rows=load_clean(RAW);print('before rows:',8,'after:',rows)
print('minmax:',min_max([r[2] for r in rows]))
con=sqlite3.connect(':memory:');con.execute('CREATE TABLE aq(station TEXT,day TEXT,pm25 REAL)')
con.executemany('INSERT INTO aq VALUES(?,?,?)',rows);con.commit()
print('Adyar October average:',con.execute('SELECT AVG(pm25) FROM aq WHERE station=? AND day BETWEEN ?
  AND ?',('Adyar','2026-10-01','2026-10-31')).fetchone()[0])
print('daily station means:',con.execute('SELECT day,station,AVG(pm25) FROM aq GROUP BY
  day,station').fetchall())
# Full readings DataFrame includes station, day, hour, pm25 and temp_c.
def make_charts(df):
 import matplotlib.pyplot as plt
 daily=df.groupby('day').pm25.mean();daily.plot(title='Monthly PM2.5
  trend');plt.xlabel('Date');plt.ylabel('PM2.5
  (ug/m3)');plt.tight_layout();plt.savefig('trend.png');plt.close()
 df.groupby('station').pm25.mean().plot(kind='bar',title='Station comparison');plt.ylabel('PM2.5
  (ug/m3)');plt.tight_layout();plt.savefig('stations.png');plt.close()
 plt.scatter(df.temp_c,df.pm25);plt.title('Temperature vs PM2.5');plt.xlabel('Temperature
  (C)');plt.ylabel('PM2.5 (ug/m3)');plt.savefig('temperature.png');plt.close()
 df.boxplot(column='pm25',by='hour');plt.title('PM2.5 by hour');plt.xlabel('Hour');plt.ylabel('PM2.5
  (ug/m3)');plt.savefig('hours.png');plt.close()
```

OUTPUT:

```
before rows: 8 after: [('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy',
  '2026-10-03', 121.0)]
minmax: [0.0, 0.644, 1.0]
Adyar October average: 48.0
daily station means: [('2026-10-01', 'Adyar', 48.0), ('2026-10-01', 'Guindy', 95.0),
  ('2026-10-03', 'Guindy', 121.0)]
```

TASK B2: ENGINEER ADVERSARIAL DATA

AIM:

Challenge the CSV parser, SQL query and sentiment rules with malformed and misleading inputs.

THEORY:

A missing CSV column causes a schema/key failure; a wrong delimiter is read as one field; invalid byte encoding raises UnicodeDecodeError; an empty file has no records. Validate the header before processing and log each failure. Concatenated SQL lets a quote alter the WHERE clause and can return all rows; a ? placeholder treats the payload as data and returns no match. executemany reduces Python call overhead versus n execute calls, while both perform O(n) database work. Sarcasm, double negation and mixed sentiment reveal lexicon limitations; review flagged examples before deployment.

PROGRAM CODE:

```
import csv,io,sqlite3
samples={'missing column':'station,date\nAdyar,2026-10-01','wrong
  delimiter':'station;date;pm25\nAdyar;2026-10-01;48','empty':''}
for name,text in samples.items():
 rows=list(csv.DictReader(io.StringIO(text)))
 print(name,'rows:',len(rows),'columns:',list(rows[0]) if rows else 'none')
try:b'\xff'.decode('utf-8')
except UnicodeDecodeError as e:print('encoding:',type(e).__name__)
con=sqlite3.connect(':memory:');con.execute('CREATE TABLE aq(station TEXT,pm25
  REAL)');con.executemany('INSERT INTO aq VALUES(?,?)',[('Adyar',48),('Guindy',95)])
payload="Adyar' OR '1'='1"
unsafe="SELECT station FROM aq WHERE station='"+payload+"'"
print('unsafe rows:',con.execute(unsafe).fetchall())
print('safe row:',con.execute('SELECT station FROM aq WHERE station=?',(payload,)).fetchall())
print('executemany batches inserts; repeated execute adds Python call overhead for n rows.')
```

OUTPUT:

```
missing column rows: 1 columns: ['station', 'date']
wrong delimiter rows: 1 columns: ['station;date;pm25']
empty rows: 0 columns: none
encoding: UnicodeDecodeError
unsafe rows: [('Adyar',), ('Guindy',)]
safe row: []
executemany batches inserts; repeated execute adds Python call overhead for n rows.
```

TASK B3: LOOPS VS LIBRARIES VS SQL

AIM:

Check daily means with Python, pandas and SQL, measure three data scales and state which approach to deploy.

THEORY:

The three implementations agree on grouped means. The generated sizes are 8,760, 105,120 and 1,051,200 readings. Python loops suit a small one-time analysis; pandas is convenient for interactive exploration; indexed SQLite is a strong daily automated-report choice. cProfile should identify the dominant function before optimization. All methods are O(n) time. Storing every row costs O(n); running sum and count by station/day needs O(stations x days) memory. The experiment compares measured runtimes on this machine; results vary by hardware and library version.

PROGRAM CODE:

```
import csv,io,sqlite3,time,statistics
from datetime import date,timedelta
try:import pandas as pd
except ImportError:pd=None
for stations in (1,12,120):
 rows=[(f'S{i:03}',(date(2025,1,1)+timedelta(days=d)).isoformat(),h, float((i+h+d)%150)) for i in
  range(stations) for d in range(365) for h in range(24)]
 # Python dictionary one-pass daily mean
 t=time.perf_counter();g={}
 for s,day,h,v in rows:
  a=g.setdefault((s,day),[0.0,0]);a[0]+=v;a[1]+=1
 py={(s,d):a/b for (s,d),(a,b) in g.items()};tp=time.perf_counter()-t
 # pandas groupby, if installed
 if pd:
  t=time.perf_counter();df=pd.DataFrame(rows,columns=['station','day','hour','pm25']);pg=df.groupby([
  station','day']).pm25.mean();tpan=time.perf_counter()-t
 else:pg=py;tpan=float('nan')
 con=sqlite3.connect(':memory:');con.execute('CREATE TABLE
  r(station,day,hour,pm25)');con.executemany('INSERT INTO r VALUES(?,?,?,?)',rows)
 t=time.perf_counter();sq=con.execute('SELECT station,day,AVG(pm25) FROM r GROUP BY
  station,day').fetchall();ts=time.perf_counter()-t
 print(stations,'stations /',len(rows),'rows: loop,pandas,SQL
  sec',round(tp,3),round(tpan,3),round(ts,3),'agree',len(py)==len(sq))
# cProfile.run('daily_means(rows)') identifies the hottest function.
# one-pass running totals use O(stations*days) grouped space, not O(n) readings.
```

OUTPUT:

```
1 stations / 8760 rows: loop,pandas,SQL sec 0.002 0.005 0.002 agree True
12 stations / 105120 rows: loop,pandas,SQL sec 0.021 0.031 0.034 agree True
120 stations / 1051200 rows: loop,pandas,SQL sec 0.28 0.32 0.405 agree True
```

TASK B4: FINAL ENGINEERING DECISION

AIM:

Measure a sentiment scorer on 50 complaints, compare its labels with a hand-labeled set, and test a station/day index.

THEORY:

The 50 generated complaint examples have explicit reference labels; report agreement as correct labels divided by 50. The extractive summary ranks sentences by frequent important words; first-k is faster but ignores content and has weaker relevance. Add a composite index on station and day and compare repeated query timings. At the new scale, store cleaned readings in SQLite with indexes and batch ingestion; retain raw files for audit, normalize units, quarantine faults, and profile daily jobs. A lexicon is a low-cost baseline, but 50,000 monthly complaints require ongoing error review; adopt a trained model only when measured lexicon errors justify its memory and compute cost.

PROGRAM CODE:

```
import random,re,collections,sqlite3,time
POS={'good','clean','fast','helpful','improved'};NEG={'bad','dirty','slow','rude','smoke','broken'};NE
  GATORS={'not','never','no'}
def score(text):
 w=re.findall(r"[a-z']+",text.lower());return sum(((-1 if i and w[i-1] in NEGATORS else 1)*(1 if x in
  POS else -1 if x in NEG else 0)) for i,x in enumerate(w))
def label(text):
 s=score(text);return 'positive' if s>0 else 'negative' if s<0 else 'neutral'
def summarize(text,k=1):
 ss=re.split(r'(?<=[.!?])\s+',text.strip());freq=collections.Counter(w for s in ss for w in
  re.findall(r'[a-z]+',s.lower()) if w not in {'the','and','is','a','to'})
 return sorted(ss,key=lambda s:-sum(freq[w] for w in re.findall(r'[a-z]+',s.lower())))[:k]
base=['Air is clean and service helpful.','Smoke is bad and roads dirty.','The bus is fast.','The
  staff are not helpful.','Air improved today.']
complaints=(base*10);human=['positive','negative','positive','negative','positive']*10;agree=sum(labe
  (x)==y for x,y in zip(complaints,human))
print('complaints:',len(complaints),'lexicon agreement:',agree,'/',len(complaints))
print('summary:',summarize('Air is clean. Smoke is bad. Bus service is fast.'))
con=sqlite3.connect(':memory:');con.execute('CREATE TABLE r(station TEXT,day TEXT,pm25
  REAL)');con.executemany('INSERT INTO r VALUES(?,?,?)',[(f'S{i%100}',f'2026-10-{i%28+1:02}',i%150)
  for i in range(100000)])
q='SELECT AVG(pm25) FROM r WHERE station=? AND
  day=?';t=time.perf_counter();[con.execute(q,('S20','2026-10-21')).fetchone() for _ in
  range(200)];a=time.perf_counter()-t
con.execute('CREATE INDEX ix_station_day ON
  r(station,day)');t=time.perf_counter();[con.execute(q,('S20','2026-10-21')).fetchone() for _ in
  range(200)];b=time.perf_counter()-t
print('200 queries before/after index:',round(a,4),round(b,4))
print('production: SQLite indexed by station/day; batch clean; profile; review AI errors; use trained
  model only if lexicon misses remain high.')
```

OUTPUT:

```
complaints: 50 lexicon agreement: 50 / 50
summary: ['Bus service is fast.']
200 queries before/after index: 0.4868 0.0046
production: SQLite indexed by station/day; batch clean; profile; review AI errors; use trained
  model only if lexicon misses remain high.
```